تحلیل و بررسی عملکرد معماری میکروسرویس در ارتباطات بین سرویسی برای انتقال جریان داده های کلان در سیستم های بلادرنگ و زمان حقیقی هوشمند

محمد امین محمدی تبار[1]

حمیدرضا مقسمی[2]

1- محمد امین محمدی تبار، دانشجوی کارشناسی ارشد مهندسی کامپیوتر ، گروه مهندسی کامپیوتر، دانشکده فنی و مهندسی، دانشگاه آزاد اسلامی واحد تهران غرب، تهران، ایران

2- حمیدرضا مقسمی، استادیار، دکتری مهندسی کامپیوتر، گروه مهندسی کامپیوتر، دانشکده فنی و مهندسی ، دانشگاه آزاد اسلامی واحد تهران غرب، تهران، ایران

چکیده

معماری میکروسرویس امروزه به عنوان یک الگوی معماری کارآمد در توسعه‌ی نرم‌افزار شناخته می‌شود. در این معماری یک برنامه‌ی بزرگ به ماژول‌های کوچک عملیاتی تبدیل می‌شود و این ماژول‌ها علی‌رغم عملکرد مستقل از هم می‌توانند با یک دیگر در ارتباط باشند. امروزه بسیاری از سازمان‌ها، شرکت‌ها و بنگاه‌های اقتصادی به منظور کاهش هزینه‌ها و بهبود عملکرد سیستم‌های نرم‌افزاری خود از نرم‌افزارهای قدیمی مبتنی بر معماری‌های مونولیتیک به معماری میکروسرویس مهاجرت می کنند. اما این کار چالش‌هایی از جمله شناسایی ماژول‌های عملیاتی و توسعه مجدد آن‌ها، استقرار و برقراری ارتباط بین سرویس ها را به‌دنبال دارد. یکی از مهم‌ترین چالش ها در معماری میکروسرویس برقراری ارتباط و انتقال داده‌ها بین سرویس های‌گوناگون است. امروزه با گسترش روزافزون استفاده از سیستم‌های مختلف نرم‌افزاری، حجم بسیار بزرگی از داده ها تولید می‌شود که انتقال این داده ها برای پردازش‌های مختلف بین میکروسرویس‌ها و برنامه‌های کاربردی مختلف که با زبان‌ها و تکنولوژی‌های مختلفی توسعه داده شده‌اند، می‌بایستی به نحوی باشد که در کارکرد سیستم‌ها اختلال ایجاد نکنند و جابجایی به صورت مطمئن و بدون از دست رفتن داده‌ها و سریع و با تاخیر حداقلی و بلادرنگ انجام شود، حال این‌که با گسترش هوش مصنوعی و یادگیری ماشین مدل های بسیاری آموزش ایجاد شده‌اند که می توانند در بسیاری از امور مثل سیستم‌های توصیه‌گر، پیش‌بینی، بهینه‌سازی و... پردازش داده انجام دهند و در صورت به کارگیری در یک سیستم که با طراحی، استراتژی و ابزارهای درست توسعه داده شده باشد، می توان یک سیستم هوشمند بلادرنگ و زمان حقیقی را پیاده سازی کرد.

کلمات کلیدی: معماری میکروسرویس، جریان داده های بزرگ، معماری رویدادمحور، ارتباطات سنکرون، ارتباطات آسنکرون، معماری رویدادمحور، جریان داده‌ها، سیستم های هوشمند، پردازش زمان حقیقی، سیستم های بلادرنگ

1.مقدمه

معماری میکروسرویس امروزه به عنوان یک پارادیم و الگوی معماری نرم افزار شناخته می‌شود. در این معماری برنامه‌های کاربردی به ماژول‌های عملیاتی کوچک تبدیل می‌شوند که ضمن کارکرد مستقل از یکدیگر با هم در ارتباط هستند. در معماری‌های مونولیتیک، تمامی اجزای یک برنامه در قالب یک برنامه‌ی واحد توسعه داده می‌شوند و پس از آن استقرار داده می‌شوند. اما در معماری میکروسرویس یک برنامه‌ی بزرگ به اجزای کوچک‌تری تبدیل می‌شود و هرکدام از این اجزا پس از

1 aminmohammaditabaramt@gmail.com
2 Moghasemi.h@wtiau.ac.ir

استقرار، ضمن عملکرد مستقل از هم با برقراری ارتباط پایدار بتوانند عملکرد مناسبی را فراهم کنند به نحوی که به مزایای بسیاری از جمله مقیاس پذیری بهتر، استفاده از تکنولوژی‌ها و زبان‌های مختلف در توسعهِی برنامه‌ها، مدیریت و کاهش هزینه‌ها منتج شود.

گرچه مهاجرت از معماری‌های مونولیتیک به معماری میکروسرویس، می تواند مزایای بسیاری را فراهم آورد، خود چالش برانگیز نیز هست، مثل شناسایی اجزای یک برنامه‌ی بسیار بزرگ و شکستن آن به تعدادی برنامه ی کوچک‌تر، توسعه نرم افزارهایی که از شکستن نرم‌افزار قبلی ساخته می‌شوند، استقرار نرم‌افزارها، پیاده سازی ارتباطات میان سرویس‌ها و چالش های دیگر در برابر مزایای پرشمار این معماری نرم‌افزاری قرار می‌گیرند اما این معماری نرم افراری علی‌رغم تمامی این چالش ها عملکرد بسیار خوب و مدیریت هزینه ها و انعطاف پذیری و مقیاس پذیری بالایی را فراهم می‌کند.

با گسترش استفاده از معماری میکروسرویس راه‌حل‌های کارآمدی برای توسعه‌ی نرم افزارها و استقرار و ارتباطات بین نرم افزارها و سرویس‌ها ارائه شده‌اند که می توانند چالش‌های مختلف را به نحو بسیار مطلوبی رفع نمایند. همچینین امروزه با پیشرفت چشم‌گیر تکنولوژی و استفاده‌ی روزافزون از سیستم‌های نرم‌افزاری، شاهد تولید چشم‌گیر داده ها در حجم بسیار بزرگ هستیم و نیاز به تبادل اطلاعات بین برنامه‌های مختلف استفاده از الگوهای‌ارتباطی را اجتناب ناپذیر می‌کند تا انواع داده‌های مختلف داده بین سیستم‌ها جابجا شود. ارتباطات و انتقال داده می‌بایستی به صورت ایمن و مطمئن با حداقل میزان تاخیر و به طور قطعی انجام شود تا اطلاعات و داده‌های تولید شده در یک برنامه به برنامه‌ی دیگر برسد و عملیات‌ها وپردازش‌های لازم انجام شود و به این ترتیب ارائه‌ی خدمات توسط سامانه‌های نرم‌افزاری به بهترین نحو ممکن انجام شود.

به طور کلی دو نوع ارتباط بین سیستم‌ها برقرار است: ارتباطات سنکرون یا همزمان و دیگری ارتباطات آسنکرون یا غیرهمزمان. هر کدام از این روش‌های برقراری ارتباط موارد کاربرد خاص خود را دارند و نمی‌توان گفت که یکی بر دیگری در تمامی موارد برتری مطلق دارد. در ادامه به شرح هرکدام خواهیم پرداخت.

ارتباطات سنکرون به نوعی از ارتباطات اطلاق می‌شود که در آن ارسال‌کننده منتظر پاسخ از دریافت‌کننده است قبل از اینکه بتواند ادامه دهد. در این نوع ارتباطات، کل فرآیند تبادل داده به صورت همزمان و به ترتیب رخ می‌دهد، به این معنی که درخواست و پاسخ باید در همان زمان انجام شوند. یکی از پرکاربردترین پروتکل‌های سنکرون در دنیای نرم‌افزار، پروتکل HTTP[1]است که معمولاً در ارتباطات وب و API های تحت وب استفاده می‌شود. REST[2] یکی از محبوب‌ترین روش‌ها برای پیاده‌سازی ارتباطات سنکرون است که از این پروتکل برای برقراری ارتباط بین سرویس‌ها و ارسال داده‌ها استفاده می‌کند. ارتباطات REST معمولاً از روش‌های HTTP مانندGET ، POST، PUT و DELETE برای ارسال درخواست‌ها و دریافت پاسخ‌ها استفاده می‌کند و داده‌ها به صورت معمول در قالب JSON یا XML منتقل می‌شوند.

گروه دیگری از پروتکل‌های سنکرون که به‌ویژه در محیط‌های پیچیده‌تر و نیازمند بهینه‌سازی‌های بیشتر استفاده می‌شود، gRPC است. gRPC[3]یک فریم‌ورک ارتباطی است که توسط گوگل توسعه یافته و بر اساس HTTP/2 عمل می‌کند. برخلاف REST که معمولاً از متن (text) برای تبادل داده‌ها استفاده می‌کند، gRPC از پروتکل باینری به نام Protocol Buffers برای انتقال داده‌ها بهره می‌برد. این امر باعث می‌شود که ارتباطات gRPC بسیار سریع‌تر و کارآمدتر از REST باشد. همچنین gRPC از ویژگی‌هایی چون جریان دوطرفه و فشرده‌سازی بهتر داده‌ها پشتیبانی می‌کند که آن را به انتخابی مناسب برای سیستم‌های نیازمند ارتباطات سریع و با حجم بالای داده تبدیل می‌کند[۱].

در ارتباطات سنکرون، علاوه بر نحوه ارسال درخواست و دریافت پاسخ، مسأله Blocking و Non-Blocking نیز مطرح است که بر نحوه عملکرد سیستم و کارایی آن تأثیر زیادی دارد. ارتباطات سنکرون به صورت Blocking هستند و ارتباطات در مدل Blocking ، هنگامی که یک درخواست ارسال می‌شود، فرایند ارسال‌کننده تا زمانی که پاسخ از سرویس مقصد دریافت نشود، منتظر می‌ماند. این بدان معناست که وقتی یک سرویس درخواست می‌فرستد، سرویس فرستنده باید منتظر دریافت پاسخ

[1] **Hyper Text Transfer Protocol**
[2] **Representational State Transfer**
[3] **Remote Procedure Calls**

از سرویس مقصد باشد تا ادامه دهد. این مدل معمولاً در پروتکل‌هایی مانند REST مشاهده می‌شود. این مدل به دلیل سادگی پیاده‌سازی بسیار محبوب است، اما از آنجا که سرویس فرستنده هیچ کاری نمی‌تواند انجام دهد تا زمانی که پاسخ دریافت کند، این باعث کاهش کارایی و مقیاس‌پذیری در شرایطی با بار بالای درخواست‌ها می‌شود [۱].

مزایای ارتباطات سنکرون شامل سادگی پیاده‌سازی و وضوح آن است. در این مدل، درخواست و پاسخ به طور مستقیم و همزمان انجام می‌شود، که پیاده‌سازی آن را نسبتاً ساده می‌کند و خطایابی و تست آن نیز آسان‌تر است. همچنین زمانی که سیستم نیاز به تعامل لحظه‌ای و پاسخ فوری دارد، ارتباطات سنکرون گزینه مناسبی است. برای مثال، در سیستم‌های پرداخت آنلاین یا تایید هویت، ارتباطات سنکرون بسیار پرکاربرد است. اما معایب اصلی این مدل شامل وابستگی زمانی است. در این مدل، فرایند کل سیستم به زمان پاسخ‌دهی از سمت سرویس مقصد وابسته است. اگر سرویس مقصد دچار تأخیر شود یا به دلایلی پاسخ‌دهی به موقع نداشته باشد، کل سیستم تحت تأثیر قرار می‌گیرد و تجربه کاربری منفی ایجاد می‌شود. علاوه بر این، اگر سرویس مقصد قطع شود یا به درستی کار نکند، سرویس ارسال‌کننده قادر به ادامه فرآیند نخواهد بود. این مشکلات به ویژه در محیط‌هایی که بار ترافیکی بالا یا درخواست‌های پیچیده دارند، می‌تواند مشکل‌ساز شود.

ارتباطات آسنکرون به نوعی از تبادل داده‌ها اطلاق می‌شود که در آن فرستنده بدون انتظار برای دریافت پاسخ از دریافت‌کننده، درخواست خود را ارسال می‌کند. در این مدل، دریافت‌کننده می‌تواند در زمان مناسب به درخواست پاسخ دهد، در حالی که فرستنده می‌تواند به کارهای دیگر خود ادامه دهد. این نوع ارتباطات در سیستم‌های توزیع‌شده و مقیاس‌پذیر کاربرد زیادی دارد، زیرا باعث می‌شود که سیستم‌ها کمتر درگیر بلاک شدن شوند و بتوانند همزمان درخواست‌ها را پردازش کنند. ارتباطات آسنکرون در مقایسه با ارتباطات سنکرون که در آن فرستنده منتظر دریافت پاسخ است، از نظر کارایی و مقیاس‌پذیری مزایای زیادی دارد. به ویژه در سیستم‌های بزرگ و پیچیده که نیاز به پردازش حجم بالایی از درخواست‌ها به صورت همزمان دارند، این نوع ارتباطات کارآمدتر است.

در این زمینه، پروتکل‌های مختلفی برای پیاده‌سازی ارتباطات آسنکرون وجود دارد که که ما در این مقاله به بررسی مهم‌ترین آن‌ها یعنی AMQP، Publish-Subscribe، Redis Pub/Sub، WebSocket می پردازیم.

پروتکل AMQP یکی از پرکاربردترین پروتکل‌ها در ارتباطات آسنکرون است. این پروتکل به طور خاص برای پیام‌رسانی بین سرویس‌ها و سیستم‌ها طراحی شده است و از قابلیت‌های پیچیده‌ای مانند صف‌بندی پیام‌ها، تحویل تضمینی و امنیت بالا برخوردار است AMQP .معمولاً در محیط‌های توزیع‌شده و سیستم‌های مالی یا حساسی که نیاز به اطمینان بالا دارند، استفاده می‌شود[۱].

مدل Publish-Subscribe (انتشار و اشتراک) نیز در بسیاری از سیستم‌های آسنکرون برای برقراری ارتباطات استفاده می‌شود. در این مدل، تولیدکنندگان پیام‌ها را به یک موضوع خاص منتشر می‌کنند و مصرف‌کنندگان به این موضوع‌ها اشتراک می‌زنند تا پیام‌ها را دریافت کنند. این مدل به طور معمول در سیستم‌هایی استفاده می‌شود که نیاز به ارسال پیام‌های بلادرنگ یا اطلاع‌رسانی دارند، مانند سیستم‌های نظارتی یا برنامه‌های چت Publish-Subscribe .این امکان را می‌دهد که مصرف‌کنندگان مختلف از یک منبع پیام به طور همزمان اطلاعات را دریافت کنند، بدون اینکه تولیدکننده یا دیگر مصرف‌کنندگان منتظر یکدیگر باشند[۲].

Redis Pub/Sub یکی دیگر از پروتکل‌های سبک‌وزن و کارآمد است که در ارتباطات آسنکرون کاربرد دارد. این پروتکل در سیستم‌های بزرگ و پیچیده که نیاز به مقیاس‌پذیری و سرعت بالا دارند، استفاده می‌شود. Redis Pub/Sub به شما این امکان را می‌دهد که پیام‌ها را بین سرویس‌ها به صورت آسنکرون ارسال کنید. این ویژگی به خصوص در برنامه‌های زمان حقیقی [1]و سیستم‌هایی که نیاز به هماهنگی سریع بین اجزای مختلف دارند، مفید است[۳] .

WebSocket نیز یک پروتکل ارتباطی است که برای ارسال داده‌ها به صورت بلادرنگ و دوطرفه طراحی شده است. این پروتکل بیشتر برای ارتباطات بلادرنگ در برنامه‌های چت، بازی‌های آنلاین و سیستم‌های مشابه که نیاز به تبادل سریع داده‌ها

[1] **Real-Time**

دارند، استفاده می‌شود. WebSocket، برخلاف HTTP که ارتباطات یک طرفه است، به کلاینت‌ها و سرورها این امکان را می‌دهد که به صورت دائمی و دوطرفه با یکدیگر در ارتباط باشند و پیام‌ها را در هر زمان ارسال و دریافت کنند. این ویژگی باعث می‌شود که WebSocket برای کاربردهایی که نیاز به ارسال داده‌های فوری و بدون تأخیر دارند، بسیار مناسب باشد [۴].

2.مروری بر ادبیات پژوهش

تکین‌دوران و همکاران [۵] به بررسی چالش‌های ارتباطات بین سرویس‌ها در معماری میکروسرویس پرداختند و راه‌حل‌هایی را برای این غلبه بر این چالش‌ها مطرح کردند. این مقاله شش چالش اصلی در ارتباطات میکروسرویس را شناسایی کرده و برای هر یک راه‌حل‌های عملی ارائه داده است که می‌توانند به توسعه‌دهندگان کمک کنند تا مشکلات موجود در پیاده‌سازی معماری میکروسرویس را برطرف کرده و سیستم‌های مقاوم‌تر و مقیاس‌پذیرتر بسازند. یکی از چالش‌های اصلی در ارتباطات میکروسرویس‌ها، تأخیر در ارسال و دریافت داده‌ها است. این تأخیر ممکن است ناشی از عواملی چون فاصله جغرافیایی، حجم زیاد داده‌ها یا پیچیدگی‌های شبکه باشد. مقاله پیشنهاد می‌کند که برای کاهش تأخیر از پروتکل‌های باینری سریع‌تر مانند gRPC به جای پروتکل‌های متنی مانند REST استفاده شود. همچنین، استفاده از کشینگ و فشرده‌سازی داده‌ها نیز می‌تواند به کاهش زمان انتقال کمک کند. چالش دیگر، امنیت در سیستم‌های توزیع‌شده است. معماری میکروسرویس‌ها به دلیل ویژگی‌های خاص خود، مشکلاتی در مدیریت امنیت ایجاد می‌کند. مقاله به این نکته اشاره می‌کند که برای رفع این چالش باید از گواهینامه‌های SSL/TLS و روش‌های احراز هویت قوی استفاده شود. علاوه بر این، شبکه‌های خصوصی مجازی[1] و سیاست‌های محدودیت دسترسی می‌توانند به بهبود امنیت کمک کنند. هماهنگی و هم‌زمانی سرویس‌ها یکی دیگر از مشکلات موجود است. در سیستم‌های توزیع‌شده، هماهنگ‌سازی وضعیت سرویس‌ها می‌تواند پیچیدگی‌هایی ایجاد کند. مقاله پیشنهاد می‌کند که برای حل این مشکل از معماری‌های رویدادمحور و مدل‌های غیرهماهنگ مانند صف‌های پیام [2] و معماری‌های Pub/Sub استفاده شود. چالش مقیاس‌پذیری نیز در سیستم‌های میکروسرویس مطرح است. با توجه به تعداد زیاد سرویس‌ها و نیاز به پردازش بارهای بالا، مقیاس‌پذیری می‌تواند یک چالش جدی باشد. برای حل این مشکل، مقاله پیشنهاد می‌کند که از کانتینرها و کلاسترها برای استقرار سرویس‌ها استفاده شود. همچنین، سیستم‌های مقیاس‌پذیر باید از مدیریت خودکار منابع بهره‌مند باشند. یکی دیگر از چالش‌ها، خطاهای ارتباطی است که می‌تواند ناشی از قطع ارتباط یا مشکلات شبکه باشد. مقاله به این نکته اشاره می‌کند که برای مدیریت این خطاها باید از مکانیزم‌هایی مانند حلقه‌های بازگشتی [3] و مکانیزم‌های تأخیر و بازنشانی استفاده شود. ثبت و مدیریت دقیق خطاها نیز به شناسایی و رفع مشکلات کمک خواهد کرد. در نهایت، پیچیدگی در مدیریت ارتباطات بین سرویس‌ها یکی دیگر از چالش‌های اساسی است. با افزایش تعداد سرویس‌ها، پیگیری وضعیت‌ها و مدیریت ارتباطات به طور مؤثر دشوار می‌شود. مقاله پیشنهاد می‌کند که از سیستم‌های کشف سرویس[4] مانند Consul و Eureka برای تسهیل فرآیند ارتباط بین سرویس‌ها استفاده شود. همچنین، انتخاب صحیح پروتکل‌های ارتباطی مناسب برای کاهش پیچیدگی می‌تواند مؤثر باشد.

بدگوجار [۶] به بررسی معماری‌های رویدادمحور یا EDA [5] برای دستیابی به تحلیل‌های بلادرنگ و کاربردهای آن در صنایع مختلف می‌پردازد. مقاله با تأکید بر اهمیت پردازش داده‌ها در لحظه، رویکردهای طراحی EDA ، فناوری‌های مرتبط و مطالعات موردی موفقیت‌آمیز در این زمینه را بررسی می‌کند. در ابتدا، مقاله مسئله عدم کارایی روش‌های پردازش دسته‌ای [6] در پاسخ به نیازهای کسب‌وکارهای مدرن را مطرح می‌کند. این روش‌ها به دلیل زمان‌بری و تأخیر، دیگر برای تحلیل‌های بلادرنگ مناسب

[1] VPN
[2] Message Queues
[3] Circuit Breakers
[4] Service Discovery
[5] Event-Driven Architectures
[6] Batch Processing

نیستند. در مقابل، معماری‌های رویدادمحور به عنوان راه‌حلی نوین و کارآمد پیشنهاد شده‌اند که قادر به پردازش و تحلیل داده‌ها به محض وقوع رویدادها هستند. نویسنده اشاره می‌کند که EDA می‌تواند به سازمان‌ها کمک کند تا با سرعت بیشتری تصمیم‌گیری کرده و به تغییرات محیطی پاسخ دهند. نمونه‌هایی از کاربردهای EDA شامل تشخیص تقلب در بانکداری، نگهداری پیش‌بینانه در تولید و بازاریابی شخصی‌سازی‌شده در تجارت الکترونیک است. روش پژوهش این مقاله شامل بررسی مولفه‌های اصلی معماری‌های رویدادمحور است که شامل تولیدکنندگان رویداد(Event Producers) ، مصرف‌کنندگان رویداد [1]، واسطه‌های رویداد [2]و منطق پردازش رویداد[3] می‌شود. همچنین مقاله به ملاحظات طراحی مانند مقیاس‌پذیری، تحمل خطا، تأخیر کم و انعطاف‌پذیری برای ایجاد سیستم‌های قوی و مقاوم اشاره دارد. فناوری‌های مطرح‌شده شامل Apache Kafka ، Apache Flink و AWS Kinesis هستند که هر کدام ویژگی‌های منحصر به فردی را ارائه می‌دهند. نتایج این پژوهش نشان می‌دهد که استفاده از EDA باعث بهبود کارایی عملیاتی، تقویت توانایی‌های تصمیم‌گیری و افزایش مزیت رقابتی می‌شود. برای مثال، در بخش بانکداری، معماری رویدادمحور به بانک‌ها کمک کرده است که با تحلیل جریان داده‌ها در زمان واقعی، الگوهای مشکوک را شناسایی کرده و هشدارهای تقلب را فوراً تولید کنند. در صنعت تولید، معماری‌های رویدادمحور از داده‌های حسگرها برای نگهداری پیش‌بینانه استفاده کرده و با کاهش زمان خرابی تجهیزات، بهره‌وری را افزایش داده‌اند. همچنین، در تجارت الکترونیک، این معماری‌ها توانسته‌اند تجربه بازاریابی شخصی‌سازی‌شده را از طریق تحلیل بلادرنگ تعاملات مشتری بهبود بخشند. نتیجه‌گیری می‌کند که پذیرش معماری‌های رویدادمحور برای سازمان‌هایی که می‌خواهند از بینش‌های بلادرنگ داده‌ها بهره‌مند شوند، ضروری است. نویسنده پیشنهاد می‌کند که سازمان‌ها باید استراتژی‌های EDA را برای ارتقاء تصمیم‌گیری و اقدامات سریع در محیط‌های پویا اتخاذ کنند. همچنین، این معماری‌ها ظرفیت بازتعریف فرآیندهای تصمیم‌گیری را در صنایع مختلف نشان داده‌اند و فرصت‌های جدیدی برای نوآوری فراهم می‌کنند.

ویراسینگ و همکاران [۷] به بررسی یک استراتژی بهینه برای کاهش تأخیر در ارتباطات بین سرویس‌ها در معماری میکروسرویس می‌پردازد. این مقاله به مسئله اصلی تأخیر در ارتباطات بین سرویس‌ها و تأثیر آن بر عملکرد کلی برنامه می‌پردازد. نویسندگان اشاره می‌کنند که یکی از چالش‌های اساسی در معماری میکروسرویس، ارتباطات میان سرویس‌ها است که به دلیل اجرای سرویس‌ها در محیط توزیع‌شده، نیازمند ارسال و دریافت بسته‌های داده از طریق شبکه است. این ارتباطات معمولاً باعث افزایش تأخیر و کاهش کارایی سیستم در مقایسه با معماری‌های مونولیتیک می‌شود. نویسندگان به دنبال ارائه راه‌حلی هستند که نه تنها این تأخیر را کاهش دهد، بلکه کارایی سیستم را از نظر زمان پاسخ‌دهی و بازدهی افزایش دهد. روش پیشنهادی این مقاله استفاده از ساختار داده‌ای Redis Streams برای پیاده‌سازی الگوی ارتباطات آسنکرون است. این راهکار بر اساس یک ارتباط مبتنی بر Pub/Sub طراحی شده است و از قابلیت‌های Redis برای انتقال پیام‌ها بین سرویس‌ها استفاده می‌کند. با استفاده از اتصال‌های سوکت TCP که در زمان راه‌اندازی سرویس ایجاد می‌شود و تا پایان اجرای آن باز می‌ماند، زمان ایجاد و بستن اتصال به طور قابل توجهی کاهش یافته است. همچنین، پیام‌ها قبل از انتقال به فرمت باینری سریالایز شده و به عنوان بایت‌بافر ارسال می‌شوند که باعث کاهش مصرف منابع شبکه و افزایش سرعت انتقال می‌شود. نتایج آزمایش‌ها نشان می‌دهد که این راهکار در مقایسه با روش استاندارد ارتباطاتHTTP ، تأخیر در ارتباطات بین سرویس‌ها را به طور چشمگیری کاهش داده است. به عنوان مثال، در آزمایش‌هایی که در محیط AWS انجام شد، این روش توانست زمان پاسخ‌دهی کلی برنامه و بازدهی آن را بهبود بخشد. مقایسه‌ها بین دو سناریو نشان داد که ارتباطات مبتنی بر Redis Streams در تمامی شرایط آزمایشی، عملکرد بهتری نسبت به روش‌های استاندارد داشت. همچنین، افزایش اندازه بسته‌های داده، تأثیر کمتری بر کارایی این راهکار در مقایسه با HTTP داشت. مقاله نتیجه‌گیری می‌کند که استفاده از Redis Streams به عنوان یک راهکار ارتباطی، می‌تواند نه تنها تأخیر را کاهش دهد، بلکه مزایای دیگری از جمله مقیاس‌پذیری بالا، ردیابی آسان، و اطمینان از تحویل پیام‌ها را فراهم

[1] **Event Consumers**
[2] **Event Brokers**
[3] **Event Processing**

کند. نویسندگان پیشنهاد می‌کنند که تحقیقات آینده بر روی توسعه راهکارهای مبتنی بر فناوری‌های بومی ابری [1]برای بهبود بیشتر عملکرد و قابلیت نگهداری سیستم متمرکز شود.

دوبلائر و همکاران [۸] به مقایسه جامع دو سیستم پرکاربرد در حوزه Publish/Subscribe یعنی Apache Kafka و RabbitMQمی‌پردازد. هدف این مقاله کمک به انتخاب سیستم مناسب برای سناریوهای مختلف است، با در نظر گرفتن تفاوت‌های معماری، ویژگی‌ها و کاربردهای این دو سیستم. مقاله مسئله‌ای را بررسی می‌کند که معمولاً توسعه‌دهندگان با آن روبرو هستند: اینکه Kafka و RabbitMQ چه تفاوت‌هایی دارند و در چه شرایطی یکی از آن‌ها انتخاب بهتری است. این دو سیستم هر دو برای مدیریت ارتباطات مبتنی بر Publish/Subscribe طراحی شده‌اند اما تفاوت‌های قابل توجهی در معماری، عملکرد و کاربردهای پیشنهادی دارند. روش پژوهش در این مقاله شامل یک چارچوب مقایسه‌ای است که ویژگی‌های اصلی این دو سیستم را از لحاظ کیفی و کمی مورد بررسی قرار می‌دهد. در بخش کیفی، مقاله به بررسی مواردی همچون تضمین‌های کیفیت خدمات(QoS) ، مقیاس‌پذیری، قابلیت اطمینان، و پیچیدگی پردازش پیام‌ها پرداخته است. در بخش کمی، عملکرد این دو سیستم از نظر تأخیر و توان عملیاتی[2] با استفاده از آزمایش‌های تجربی مقایسه شده است. نتایج پژوهش نشان می‌دهد که Kafkaبرای سناریوهایی که نیاز به مقیاس‌پذیری بالا و ذخیره طولانی‌مدت پیام‌ها دارند، انتخاب بهتری است. ویژگی‌هایی مانند ذخیره پیام‌ها بر روی دیسک، قابلیت بازپخش پیام‌ها، و مقیاس‌پذیری بر اساس پارتیشن‌بندی، Kafka را برای پردازش حجم بالای داده‌ها مناسب می‌سازد. از سوی دیگر، RabbitMQ برای سناریوهایی که به تأخیر پایین، مسیریابی پیچیده پیام‌ها، و پشتیبانی از پروتکل‌های متعدد نیاز دارند، توصیه شده است. ویژگی‌هایی مانند کنترل جریان پیام، پشتیبانی از پروتکل‌های MQTTوAMQP و ابزارهای مدیریت و مانیتورینگ قوی، RabbitMQ را برای کاربردهای تعاملی و بلادرنگ مناسب می‌کند. در نهایت، مقاله نتیجه‌گیری می‌کند که انتخاب بین Kafka و RabbitMQ باید بر اساس نیازمندی‌های خاص سیستم انجام شود. برای برخی از سناریوها، استفاده ترکیبی از این دو سیستم می‌تواند بهترین راه‌حل باشد. به عنوان مثال، RabbitMQ می‌تواند برای مدیریت پیام‌های بلادرنگ در لایه اول استفاده شود و Kafka برای ذخیره‌سازی طولانی‌مدت و پردازش داده‌ها در لایه دوم. نویسندگان پیشنهاد می‌کنند که توسعه‌دهندگان قبل از انتخاب هر یک از این سیستم‌ها، جنبه‌های مختلف نیازمندی‌های خود را به دقت ارزیابی کنند.

اسلوتا و همکاران [۹] به بهینه‌سازی Apache Kafka برای پردازش جریان داده در سیستم‌هایی که به تأخیر حساس هستند می‌پردازد. این مقاله به مسئله‌ی تأخیر و مصرف منابع در سیستم‌هایی که از Kafka به‌عنوان یک سیستم پیام‌رسان استفاده می‌کنند، می‌پردازد. نویسندگان اشاره می‌کنند که بسیاری از سیستم‌های حساس به تأخیر مانند خدمات پیشنهاددهی، شبکه‌های حسگر و سیستم‌های تبلیغات بلادرنگ (Real-Time Bidding) نیاز به پردازش داده‌های جریان به‌صورت همزمان و با تأخیر بسیار پایین دارند. با این حال، اضافه کردن KafkaProducer به این سیستم‌ها می‌تواند تأثیر منفی بر مصرف منابع مانند CPU و حافظه داشته باشد و در نهایت باعث افزایش تأخیر شود. این مقاله به دنبال کاهش این تأثیرات از طریق بهینه‌سازی است. روش پیشنهادی این مقاله شامل اجرای مجموعه‌ای از آزمایش‌های عملکردی برای اندازه‌گیری تأثیر KafkaProducer بر تأخیر و مصرف منابع سیستم است. این آزمایش‌ها بر روی یک محیط تولید جداگانه انجام شده‌اند و از ابزارهایی مانند HdrHistogram برای اندازه‌گیری دقیق تأخیر و مصرف منابع استفاده شده است. نویسندگان از دو تکنیک اصلی برای بهبود عملکرد استفاده کرده‌اند: کاهش تعداد فراخوانی‌های API و استفاده از بافرهای بزرگ‌تر برای ارسال پیام‌ها. نتایج نشان می‌دهد که استفاده از بافرهای ۵۱۲ کیلوبایتی در KafkaProducer بهترین عملکرد را از نظر کاهش تأخیر و مصرف منابع ارائه می‌دهد، به‌طوری که مصرف CPU تنها ۴.۲٪ افزایش یافته است. همچنین، نسخه اصلاح‌شدهKafkaProducer که از تخصیص حافظه کارآمدتری استفاده می‌کند، توانسته است مصرف CPU را به ۳.۶٪ کاهش دهد. این بهینه‌سازی‌ها نشان می‌دهند که می‌توان با تنظیم دقیق پارامترها و استفاده از تکنیک‌های بهینه‌سازی، تأثیرات منفی Kafka را به حداقل رساند. مقاله نتیجه‌گیری می‌کند

[1] **Cloud-Native**
[2] **Throughput**

که با استفاده از این روش‌های بهینه‌سازی، سیستم‌های حساس به تأخیر می‌توانند از Kafka برای پردازش جریان داده بهره‌مند شوند بدون اینکه عملکرد کلی سیستم به خطر بیفتد. نویسندگان پیشنهاد می‌کنند که در تحقیقات آتی، این روش‌ها برای سایر سیستم‌های پیام‌رسان و محیط‌های تولیدی مقایسه شوند تا اثرات بهینه‌سازی‌های مشابه بررسی شوند.

حسین و همکاران [۱۰] به ارزیابی و مقایسه چهار سیستم پیام‌رسانی معروفKafka ، Pulsar، Artemis، و RocketMQ می‌پردازد. هدف اصلی مقاله بررسی عملکرد این سیستم‌ها در سناریوهای مختلف و ارائه معیارهای کلیدی برای انتخاب بهترین گزینه بر اساس نیازهای سیستم است. مقاله به مسئله انتخاب سیستم پیام‌رسانی مناسب در معماری‌های توزیع‌شده می‌پردازد، جایی که ارتباطات آسنکرون و جداسازی اجزای سیستم اهمیت زیادی دارد. هر یک از این سیستم‌ها ویژگی‌های منحصر به فردی دارند که آن‌ها را برای استفاده در شرایط خاص مناسب می‌سازد. به عنوان مثال، Kafkaبرای توان عملیاتی بالا و مقیاس‌پذیری طراحی شده است، در حالی که Pulsar با قابلیت‌هایی مانند چنداجاره‌ای [1]و ذخیره‌سازی لایه‌ای برجسته می‌شود. Artemis به دلیل معماری سبک و قابل تنظیم خود، و RocketMQ با تمرکز بر تحمل خطا و ادغام گسترده، مورد توجه قرار گرفته‌اند. برای ارزیابی این سیستم‌ها، مقاله از ابزار OpenMessaging Benchmark Framework استفاده کرده است که توانایی بررسی تأخیر، توان عملیاتی، استفاده از CPU ، مصرف حافظه، و کارایی جمع‌آوری زباله[2] را فراهم می‌کند. نتایج نشان می‌دهد که هر سیستم رفتار متفاوتی در داخل محیط JVM [3]دارد و این رفتار تأثیر قابل توجهی بر عملکرد کلی آن‌ها دارد. نتایج نشان می‌دهد که Kafka در توان عملیاتی بسیار قوی عمل می‌کند و مناسب برای پردازش حجم بالای پیام‌ها است. با این حال، مصرف منابع بیشتری از جمله CPU و حافظه دارد Artemis .عملکرد بسیار خوبی در کاهش تأخیر دارد، به ویژه در پیام‌های با اندازه کوچک Pulsar .با مصرف کم منابع، گزینه‌ای متعادل‌تر است اما تأخیر بیشتری در پیام‌های با اندازه بزرگ دارد. RocketMQ در سناریوهای با حجم پیام پایین عملکرد مناسبی دارد اما در مواجهه با بارهای سنگین‌تر محدودیت‌هایی از خود نشان می‌دهد. مقاله نتیجه‌گیری می‌کند که انتخاب سیستم پیام‌رسانی باید بر اساس نیازهای خاص برنامه و محدودیت‌های منابع صورت گیرد. همچنین، پیشنهاد می‌شود که تحقیقات بیشتری در زمینه معیارهای دیگر مانند ورودی/خروجی دیسک و شبکه، امنیت، و قابلیت همکاری انجام شود تا درک بهتری از این سیستم‌ها در شرایط مختلف فراهم شود.

شفابخش و همکاران [۱۱] این مقاله توسط بنیامین شافه‌بخش و همکارانش نوشته شده و به بررسی تأثیر روش‌های ارتباط بین‌فرآیندی در معماری میکروسرویس پرداخته است. نویسندگان به تأثیر مستقیم انتخاب روش ارتباطی بر عملکرد و دسترس‌پذیری سیستم‌های میکروسرویس اشاره کرده و دو سبک اصلی ارتباطی، یعنی سنکرون مانند REST ، gRPC و آسنکرون مانند RabbitMQ را مورد مقایسه قرار داده‌اند. هدف مقاله این است که نشان دهد هر کدام از این روش‌ها در چه شرایطی بهتر عمل می‌کنند. برای ارزیابی این موضوع، یک سناریوی شبیه‌سازی‌شده برای یک سیستم تجارت الکترونیک طراحی شد که شامل چندین میکروسرویس مستقل مانند خدمات محصول، نظرات مشتریان، توصیه‌ها، حمل‌ونقل و سبد خرید مشتریان است. این میکروسرویس‌ها با استفاده از Node.js توسعه داده شدند و از پایگاه‌های داده MongoDB و MySQL برای ذخیره‌سازی اطلاعات بهره بردند. همه میکروسرویس‌ها در کانتینرهای Docker و در یک کلاستر Kubernetes اجرا شدند. ارتباطات بین میکروسرویس‌ها با روش‌های سنکرون REST و gRPC و آسنکرون RabbitMQ پیاده‌سازی شد و آزمایش‌هایی برای بررسی کارایی و دسترس‌پذیری انجام گرفت. نتایج نشان داد که در شرایط بار پایین، روش‌های سنکرون، به‌ویژهgRPC ، توان عملیاتی بالاتری ارائه می‌دهند. اما با افزایش بار، روش آسنکرون RabbitMQ عملکرد بهتری نشان داد و توانست درخواست‌های بیشتری را پردازش کند. از نظر دسترس‌پذیری، RabbitMQ به دلیل ساختار جداسازی قوی‌تر و استفاده از صف پیام، برتری داشت. در شرایطی که قطع کوتاه‌مدت سرویس رخ می‌داد، روش آسنکرون تأثیر کمتری بر کاربران داشت، در حالی که روش‌های سنکرون به‌شدت تحت تأثیر خرابی‌ها قرار می‌گرفتند. این مطالعه نشان می‌دهد که انتخاب روش ارتباط

[1] **Multi-Tenancy**
[2] **Garbage Collection**
[3] **Java Virtual Machine**

بین‌فرآیندی باید بر اساس نیازهای خاص سیستم و سرویس‌ها انجام شود. در مواقعی که پاسخ‌دهی سریع و بلادرنگ مورد نیاز است، روش‌های سنکرون انتخاب بهتری هستند، اما برای سیستم‌هایی که بار زیادی را پردازش می‌کنند یا به مقیاس‌پذیری بالایی نیاز دارند، روش‌های آسنکرون مناسب‌ترند. نویسندگان توصیه می‌کنند که طراحی معماری میکروسرویس‌ها با دقت و با توجه به نیازهای هر سرویس انجام شود.

خریجی و همکاران [۱۲] به طراحی و پیاده‌سازی معماری رویدادمحور مبتنی بر ابر (REDA) برای پردازش داده‌های بلادرنگ در شبکه‌های حسگر بی‌سیم می‌پردازد. مسئله‌ای که مقاله مطرح می‌کند، نیاز به معماری‌ای است که بتواند حجم عظیمی از داده‌های تولیدشده توسط حسگرهای بی‌سیم را با تأخیر کم و قابلیت اطمینان بالا پردازش کند. چالش‌های مطرح‌شده شامل اشباع پهنای باند شبکه، تأخیر زیاد ناشی از روش‌های ذخیره‌سازی سنتی داده‌ها، و هزینه بالای راه‌اندازی زیرساخت‌های ابری است. نویسندگان پیشنهاد کرده‌اند که با استفاده از معماری REDA که مبتنی بر سیستم‌های رویدادمحور است، این مشکلات کاهش یابد. روش ارائه‌شده در این مقاله شامل استفاده از پروتکل پیام‌رسانی سبک MQTT برای انتقال داده‌های حسگرها به ابر و پردازش داده‌ها با استفاده از Kafka Streams است REDA .از الگوی طراحی میکروسرویس برای کاهش پیچیدگی سیستم استفاده می‌کند و هر میکروسرویس وظیفه خاصی را بر عهده دارد. برای اطمینان از عملکرد بالا، کلاستر Kafkaهمراه با Zookeeper در سه منطقه جغرافیایی مختلف مستقر شده است. همچنین، داده‌ها با استفاده از MongoDB ذخیره می‌شوند که قابلیت مقیاس‌پذیری و دسترس‌پذیری بالایی را ارائه می‌دهد. نتایج آزمایش‌ها نشان می‌دهد که این معماری می‌تواند تا ۸۰۰۰ پیام در ثانیه را با تأخیر بسیار کم پردازش کند. بهینه‌سازی‌هایی مانند افزایش تعداد پارتیشن‌هایKafka ، دسته‌بندی پیام‌ها توسط تولیدکننده، و استفاده از چندین رشته کاری برای مصرف‌کننده‌ها به عملکرد بهتر معماری کمک کرده است. همچنین، تست‌های بارگذاری نشان داده‌اند که REDA در شرایط استرس بالا پایدار باقی می‌ماند. در نهایت نیز بیان می‌شود که REDA با ترکیب فناوری‌های متن‌باز و بهره‌گیری از طراحی بهینه، راه‌حل موثری برای پردازش بلادرنگ داده‌های حسگرها ارائه می‌دهد. نویسندگان پیشنهاد می‌کنند که در آینده ویژگی‌های بیشتری مانند امنیت، مقیاس‌پذیری بیشتر، و استفاده از فناوری محاسبات مه[1] به معماری افزوده شود.

لِکالا [۱۳] به طراحی و پیاده‌سازی کلاسترهای کافکا با عملکرد بالا و مقیاس‌پذیری برای پردازش داده‌های بلادرنگ می‌پردازد. نویسنده به بررسی چالش‌های مرتبط با راه‌اندازی و مدیریت کلاسترهای کافکا و همچنین بهینه‌سازی آن‌ها برای کارایی بهتر پرداخته است. این مقاله مسئله رشد سریع حجم داده‌ها در سیستم‌های امروزی را مطرح کرده و بر اهمیت پردازش بلادرنگ داده‌ها برای تصمیم‌گیری سریع در سازمان‌ها تأکید دارد. کافکا به‌عنوان یک پلتفرم توزیع‌شده و متن‌باز، برای پردازش حجم بالای داده‌ها طراحی شده است، اما راه‌اندازی کلاسترهایی با عملکرد بالا و مقیاس‌پذیری بالا همچنان چالش‌برانگیز است. نویسنده پیشنهاد می‌کند که طراحی معماری کلاستر کافکا باید با توجه به نیازهای مقیاس‌پذیری و تحمل خطا انجام شود. مقاله به بررسی چندین استراتژی از جمله پارتیشن‌بندی داده‌ها، تکرار داده‌ها، تنظیمات بهینه‌سازی و یکپارچه‌سازی با سایر فناوری‌های داده‌محور می‌پردازد. روش‌های پارتیشن‌بندی مانند "هش‌پارتیشنینگ" و "راند-رابین پارتیشنینگ" برای افزایش موازی‌سازی پردازش داده‌ها استفاده می‌شوند. همچنین، استفاده از تکنیک‌های تکرار داده‌ها [۲]برای اطمینان از پایداری و تحمل خطا در کلاسترها توصیه شده است. مقاله نتیجه می‌گیرد که بهینه‌سازی کلاسترهای کافکا با تنظیماتی مانند افزایش اندازه دسته پیام‌ها، زمان‌بندی مناسب برای ارسال پیام‌ها، و استفاده از الگوریتم‌های فشرده‌سازی می‌تواند کارایی سیستم را افزایش دهد. همچنین، بهره‌گیری از ابزارهایی مانند Apache Spark و Hadoop برای یکپارچه‌سازی کافکا با سایر فناوری‌های پردازش داده به‌عنوان راهکاری مؤثر برای تحلیل داده‌های بلادرنگ معرفی شده است. نویسنده تأکید دارد که طراحی و مدیریت کلاسترهای کافکا باید با استفاده از بهترین شیوه‌ها و فناوری‌های پیشرفته انجام شود تا سازمان‌ها بتوانند از قدرت پردازش

[1] **Fog Computing**
[2] **Replication**

داده‌های بلادرنگ بهره‌مند شوند و تصمیم‌گیری‌های سریع و هوشمندانه‌ای انجام دهند. پیشنهاداتی نیز برای استفاده از فرآیندهای Failover و استقرار چندکلاستر برای افزایش قابلیت اطمینان و کاهش خطرات ارائه شده است.

بندی و هورتادو [۱۴] به طراحی معماری داده‌های استریم برای پردازش بلادرنگ در محیط‌های محاسبات لبه‌ای می‌پردازد. این پژوهش از معماری Kappa برای پردازش داده‌های بلادرنگ با استفاده از ابزارهایی مانند Apache Kafka و Rockset بهره می‌برد و هدف آن کاهش تأخیر در انتقال داده‌ها از منابع (مانند توییتر) به مقصد، مانند ابزار Tableau برای مصورسازی است. مقاله به مسئله رشد سریع داده‌های تولیدی و چالش‌های مربوط به ذخیره‌سازی و پردازش آن‌ها می‌پردازد. در محیط‌های محاسبات لبه‌ای، انتقال حجم عظیمی از داده‌ها به فضای ابری می‌تواند باعث افزایش تأخیر و کاهش کارایی شود. نویسندگان پیشنهاد می‌کنند که به جای ذخیره و بازیابی داده‌ها، باید از معماری‌هایی بهره برد که داده‌ها را قبل از انتقال پردازش و بهینه‌سازی کنند. روش ارائه‌شده شامل پیاده‌سازی یک سیستم پیام‌رسانی انتها به انتها برای استخراج داده‌های توییتر، پردازش آن‌ها از طریق Kafka و انتقال داده‌های پردازش‌شده به Rockset و سپس Tableau است. توییتر به‌عنوان منبع داده‌های بلادرنگ عمل می‌کند و Kafka برای انتقال و پردازش پیام‌ها استفاده می‌شود. Rockset به‌عنوان یک ابزار تجزیه‌وتحلیل بلادرنگ برای ایجاد شاخص‌ها و کوئری‌های سریع عمل می‌کند. در نهایت، داده‌ها در Tableau مصورسازی می‌شوند. نتایج نشان می‌دهد که این معماری می‌تواند داده‌های توییتر را هر ۳۰ ثانیه پردازش و به Tableau منتقل کند. بهینه‌سازی‌های انجام‌شده، شامل استفاده از Kafka برای مدیریت داده‌ها در حجم بالا و کاهش تأخیر به دو میلی‌ثانیه در پردازش پیام‌ها، عملکرد این سیستم را قابل‌قبول کرده است. از طرفی، استفاده از ابزار Rockset باعث شده که داده‌ها به صورت بلادرنگ فیلتر و تحلیل شوند. در نتیجه‌گیری، مقاله به اهمیت معماری‌های داده‌محور برای کاهش تأخیر و افزایش سرعت پردازش در محیط‌های لبه‌ای اشاره می‌کند و پیشنهاد می‌دهد که در تحقیقات آینده، از الگوریتم‌های یادگیری ماشین برای تحلیل پیشرفته‌تر داده‌ها و پیش‌بینی احساسات در داده‌های شبکه‌های اجتماعی استفاده شود.

مارتین و همکاران [۱۵] به معرفی یک چارچوب متن‌باز برای مدیریت پایپ‌لاین‌های یادگیری ماشین[1] و هوش مصنوعی[2] از طریق جریان داده می‌پردازد. این مقاله به مسئله‌ی عدم تطابق چارچوب‌های ML/AI با جریان‌های داده‌ی مداوم می‌پردازد. چارچوب‌های رایج معمولاً برای کار با داده‌های ایستا طراحی شده‌اند و از جریان داده پشتیبانی محدودی دارند. این چالش در عصر داده‌های کلان و اینترنت اشیا [3]پررنگ‌تر شده است، جایی که جریان‌های داده به‌شدت در حال گسترش‌اند. نویسندگان با معرفیKafka-ML ، یک چارچوب متن‌باز، سعی کرده‌اند این فاصله را کاهش دهند Kafka-ML .به کاربران اجازه می‌دهد مدل‌های ML را طراحی، آموزش، ارزیابی و برای پیش‌بینی مستقر کنند. این چارچوب از Kafka به‌عنوان سیستم پیام‌رسانی برای انتقال و مدیریت جریان داده استفاده می‌کند. تمامی اجزای این چارچوب با استفاده از Docker کانتینری شده‌اند و از Kubernetes برای مدیریت و مقیاس‌پذیری بهره می‌برد Kafka-ML .یک رابط کاربری ساده و کاربرپسند ارائه می‌دهد که برای متخصصان و کاربران تازه‌کار مناسب است. روش پیشنهادی شامل یک معماری میکروسرویسی است که هر جزء وظیفه‌ی مشخصی دارد Kafka-ML .از فرمت‌های داده مختلف مانند RAW و Avro برای پردازش جریان‌های داده پشتیبانی می‌کند و با TensorFlow یکپارچه است. این چارچوب قابلیت‌هایی نظیر مدیریت خطا، تحمل خطا، و مقیاس‌پذیری بالا را ارائه می‌دهد. نتایج ارزیابی‌ها نشان می‌دهد که Kafka-ML می‌تواند با بهره‌گیری از ویژگی‌هایی مانند کنترل پیام‌ها و سیاست‌های نگهداری داده‌هایKafka ، مدیریت داده‌ها و پایپ‌لاین‌های ML را بهبود بخشد. آزمایش‌ها نشان دادند که این سیستم قادر به مدیریت کارآمد جریان داده‌ها حتی در شرایط بارگذاری بالا است و از طریق Kafka می‌تواند داده‌ها را بدون نیاز به ذخیره‌سازی جداگانه مدیریت کند. مقاله پیشنهاد می‌کند که در آینده این چارچوب با چارچوب‌های دیگر یادگیری ماشین یکپارچه شود و قابلیت‌هایی مانند یادگیری آنلاین و بهینه‌سازی هایپرامترها به آن افزوده شود Kafka-ML .به‌عنوان یک ابزار متن‌باز و آماده برای تولید، مسیر جدیدی را برای یکپارچگی جریان‌های داده و یادگیری ماشین معرفی کرده است.

---

[1] ML
[2] AI
[3] IoT

کول و همکارانش [۱۶] به طراحی و پیاده‌سازی یک سیستم شناسایی وسایل نقلیه بلادرنگ بر اساس ویژگی‌های نوع، رنگ و سرعت با استفاده از معماری رویدادمحور و Kafka Streams می‌پردازد. مقاله مسئله‌ی بهبود سیستم‌های مدیریت ترافیک و نظارت بر آن را بررسی می‌کند. این موضوع چالش‌هایی مانند نیاز به مقیاس‌پذیری، تحمل خطا، و پاسخ‌دهی سریع را مطرح می‌کند. رویکرد ارائه‌شده از ترکیب معماری‌های ارتباطات سنکرون و آسنکرون در چارچوب میکروسرویس استفاده می‌کند و هدف آن بهبود پردازش جریان‌های ویدیویی و کاهش تأخیر در بازیابی داده‌ها است. در این سیستم، دوربین‌های نظارتی داده‌ها را به میکروسرویس‌هایی ارسال می‌کنند که این داده‌ها را بر اساس نوع وسیله، رنگ و سرعت فیلتر کرده و در قالب موضوعات خاص منتشر می‌کنند. میکروسرویس‌های ترکیبی این موضوعات را ادغام کرده و موضوعات جدیدی مانند نوع-رنگ یا نوع-رنگ-سرعت ایجاد می‌کنند. این رویکرد امکان جستجوی کارآمدتر و بلادرنگ را فراهم می‌آورد و کاربران می‌توانند بدون نیاز به پردازش دوباره ویدیوها، نتایج مرتبط با پرسش خود را دریافت کنند. نتایج آزمایش‌ها نشان می‌دهد که سیستم پیشنهادی با استفاده از YOLOv4 و نسخه سبک‌تر آن (Tiny-YOLOv4) در شناسایی نوع و سرعت وسایل نقلیه عملکرد خوبی داشته و دقتی بالاتر از ۹۸ درصد را ارائه داده است. همچنین، از CNN برای تشخیص رنگ استفاده شده که با استفاده از فضای رنگی HSV به دقت بالایی دست یافته است. Kafka به‌عنوان یک سیستم پیام‌رسانی مرکزی عمل کرده و ارتباط بین میکروسرویس‌ها را مدیریت می‌کند. این مطالعه تأکید می‌کند که ترکیب معماری میکروسرویس، Kafka و الگوریتم‌های یادگیری عمیق می‌تواند بهبود قابل‌توجهی در سیستم‌های نظارت بر ترافیک و مدیریت آن ایجاد کند.

3.یافته‌های پژوهش

ارتباطات بین سرویس‌ها در معماری میکروسرویس یکی از نقاط محوری عملکرد سیستم‌های توزیع‌شده است. یکی از بزرگ‌ترین چالش‌ها در این زمینه، مدیریت تأخیر در ارتباطات است. ابزارهایی که بر اساس مدل‌های سنکرون کار می‌کنند، مانند REST و gRPC ، برای سناریوهایی طراحی شده‌اند که در آن‌ها زمان پاسخ‌دهی سریع مورد نیاز است. این ابزارها در شرایط بار پایین، عملکرد خوبی دارند و می‌توانند برای پاسخ‌های بلادرنگ و فوری استفاده شوند. با این حال، هنگامی که تعداد درخواست‌ها افزایش می‌یابد، این مدل به دلیل وابستگی مستقیم بین درخواست و پاسخ، دچار تأخیر می‌شود. برای مثال، REST به دلیل استفاده از پروتکل HTTP و مدل متنی خود، در محیط‌های با بار بالا ناکارآمد است. در مقابل، gRPC با بهره‌گیری از HTTP/2 و پیام‌های باینری، توانسته عملکرد بهتری ارائه دهد، اما همچنان در بارهای سنگین و نیاز به مقیاس‌پذیری گسترده محدودیت‌هایی دارد.

ابزارهای آسنکرون مانند RabbitMQ و Kafka با استفاده از مدل انتشار-اشتراک و صف‌های پیام، به‌طور مؤثری تأخیر را کاهش داده‌اند. Kafka، به‌ویژه در سناریوهایی که حجم بالای داده باید پردازش شود، توانسته تأخیر را به حداقل رسانده و سرعت پردازش را افزایش دهد. RabbitMQ نیز در محیط‌هایی با تعاملات بلادرنگ و سبک، مانند چت آنلاین یا اطلاع‌رسانی‌های سریع، به‌خوبی عمل کرده است. اما در مقایسه باKafka ، برای مدیریت حجم بسیار بالای داده‌ها کارایی کمتری دارد.

چالش دیگر در معماری میکروسرویس، مقیاس‌پذیری است که یکی از ویژگی‌های اصلی سیستم‌های توزیع‌شده به‌شمار می‌رود. Kafka به دلیل معماری مبتنی بر پارتیشن‌بندی و قابلیت تکرار داده‌ها، به یکی از بهترین گزینه‌ها برای محیط‌های با مقیاس بزرگ تبدیل شده است. درKafka ، هر پارتیشن می‌تواند به صورت مستقل پردازش شود، که این امر امکان پردازش همزمان حجم زیادی از داده‌ها را فراهم می‌کند. این ویژگی باعث شده Kafka در سناریوهایی مانند تحلیل بلادرنگ داده‌های شبکه‌های اجتماعی یا پایش وضعیت حسگرهای

IoT، بسیار مؤثر باشد. در مقابل، RabbitMQ به دلیل طراحی ساده‌تر و نیاز به مدیریت پیچیده‌تر برای مقیاس‌پذیری، معمولاً در محیط‌های کوچک‌تر و با نیاز به تعاملات سریع‌تر مورد استفاده قرار می‌گیرد.

امنیت یکی دیگر از چالش‌های کلیدی در ارتباطات بین سرویس‌ها است. در ابزارهای سنکرون مانند REST، امنیت به شدت به استفاده از پروتکل HTTPS وابسته است. اگرچه HTTPS برای رمزنگاری داده‌ها و حفاظت از آن‌ها در برابر حملات رایج مانند حملات مرد میانی (MITM) مؤثر است، اما مدیریت و پیکربندی آن می‌تواند پیچیده باشد. Kafka و RabbitMQ با پشتیبانی از رمزنگاری داده‌ها، احراز هویت کاربران، و سیاست‌های کنترل دسترسی [1] قابلیت‌های پیشرفته‌تری در این زمینه ارائه داده‌اند. Kafka با امکان استفاده از گواهینامه‌های SSL/TLS برای تمامی ارتباطات و پشتیبانی از پروتکل SASL، امنیتی قابل‌اطمینان را در محیط‌های توزیع‌شده فراهم می‌کند.

نام ابزار	موارد کاربرد	نقاط قوت	نقاط ضعف
Kafka	**پردازش داده‌های بلادرنگ، استریم‌سازی رویدادها، تجمیع لاگ‌ها، معماری‌های رویدادمحور.**	**مقیاس‌پذیری بالا، تحمل خطا، پشتیبانی از حجم بالای داده‌ها، پایداری در شرایط بار سنگین**	**مصرف بالای منابع ، مناسب نبودن برای سناریوهای با تأخیر بسیار پایین**
Pulsar	**محیط‌های چند اجاره‌ای، برنامه‌های بومی ابری، سیستم‌های ذخیره‌سازی لایه‌ای**	**پشتیبانی از چند اجاره‌ای، بهینه‌شده برای محیط‌های ابری، ذخیره‌سازی پویا و انعطاف‌پذیری بالا**	**تأخیر بیشتر برای پیام‌های بزرگ، کمتر بهینه برای توان عملیاتی بسیار بالا**
Artemis	**سیستم‌های با تحمل خطای بالا، یکپارچه‌سازی با برنامه‌های توزیع‌شده، حجم پیام کم**	**تحمل خطای قوی، سازگاری بالا با برنامه‌های توزیع‌شده، مناسب برای کاربردهای خاص**	**عملکرد محدود در شرایط بار سنگین، امکانات پیشرفته کمتر نسبت به رقبا**
Redis Streams	**رتباطات آسنکرون، صف‌بندی پیام‌ها، ارتباطات بین‌سرویسی سبک**	**سبک و ساده برای پیام‌رسانی، مناسب برای معماری‌های رویدادمحور و پردازش سبک**	**محدودیت در پایداری داده‌ها، کمتر بهینه برای سناریوهای با توان عملیاتی بالا نسبت به Kafka**
RabbitMQ	**پیام‌رسانی بلادرنگ، برنامه‌های تعاملی، پشتیبانی از پروتکل‌های متنوع مانند AMQP و MQTT**	**پشتیبانی از پروتکل‌های گسترده، ابزارهای قوی برای مدیریت و نظارت، مناسب برای برنامه‌های بلادرنگ**	**پیچیدگی بیشتر در مدیریت برای استقرارهای بزرگ‌مقیاس، مناسب نبودن برای توان عملیاتی بسیار بالا**
WebSocket	**ارتباط بلادرنگ دوطرفه برای چت، بازی‌های آنلاین و اعلان‌ها**	**پروتکل سبک و پایدار برای تعاملات بلادرنگ، پشتیبانی از تأخیر پایین**	**مناسب نبودن برای پایداری پیام یا محیط‌هایی با تولیدکننده‌های چندگانه**

جدول 1- مقایسه بین موارد کابرد ابزارهای به کارگرفته‌شده و نقاط ضعف و قوت آن‌ها

تحمل خطا و مدیریت خطاها نیز از جمله مسائلی است که مقالات به آن پرداخته‌اند. در معماری‌های توزیع‌شده، وقوع خطاها اجتناب‌ناپذیر است، و ابزارهای آسنکرون در این زمینه مزایای قابل‌توجهی دارند. Kafka با تکرار داده‌ها در پارتیشن‌های مختلف، از از دست رفتن داده‌ها جلوگیری کرده و امکان بازیابی سریع داده‌ها در صورت خرابی را فراهم می‌کند. RabbitMQ نیز با استفاده از صف‌های جداگانه و قابلیت‌های بازگرداندن پیام[2]،

[1] **ACL**
[2] **Message Acknowledgment**

توانسته مدیریت خطاها را بهبود بخشد. در مقابل، ابزارهای سنکرون مانند gRPC به دلیل وابستگی مستقیم میان سرویس‌ها، در صورت وقوع خطا در یک سرویس، کل سیستم را تحت تأثیر قرار می‌دهند.

یکی از چالش‌های پیچیده‌تر، مدیریت جریان داده‌های بلادرنگ است که در محیط‌هایی مانند محاسبات لبه‌ای یا تحلیل داده‌های کلان اهمیت ویژه‌ای دارد Kafka .در این زمینه به دلیل معماری خاص خود، توانسته نقش مهمی ایفا کند. با استفاده از Kafka Streams ، داده‌ها به‌صورت مستقیم از منبع پردازش می‌شوند و این امر باعث کاهش تأخیر و افزایش کارایی سیستم شده است. از سوی دیگر، Rockset نیز با ارائه ابزارهایی برای تحلیل بلادرنگ داده‌ها، به‌ویژه در زمینه‌های تجزیه‌وتحلیل شبکه‌های اجتماعی، گزینه‌ای مؤثر بوده است.

در مجموع، ابزارها و روش‌های بررسی‌شده نشان می‌دهند که انتخاب ابزار مناسب، وابسته به نوع چالش و نیازمندی‌های خاص هر سیستم است. ابزارهای سنکرون برای پاسخ‌دهی سریع و محیط‌های سبک مناسب‌ترند، در حالی که ابزارهای آسنکرون در مدیریت بارهای سنگین و چالش‌های مقیاس‌پذیری و تحمل خطا، عملکرد بهتری ارائه می‌دهند. طراحی معماری میکروسرویس باید با در نظر گرفتن این ویژگی‌ها و با توجه به نیازهای عملیاتی هر سیستم انجام شود. این تحلیل می‌تواند مبنای محکمی برای ارائه پیشنهادات و تحقیقات آینده در زمینه ارتباطات بین‌سرویسی فراهم کند.

چالش	بهترین ابزار	توضیح
تاخیر	**Kafka, RabbitMQ**	**کاهش تأخیر با استفاده از صف پیام و مدل Pub/Sub**
مقیاس پذیری	**Kafka**	**پارتیشن‌بندی و تکرار داده‌ها، مناسب برای بارهای سنگین**
امنیت	**Kafka, RabbitMQ**	**پشتیبانی از مکانیزم‌های امنیتی پیشرفته**
تحمل خطا	**Kafka, RabbitMQ**	**معماری توزیع‌شده و مدیریت پیام‌ها**
مدیریت سیستم های بلادرنگ	**Kafka, Rockset**	**پردازش کارآمد داده‌های بلادرنگ.**

جدول 2- مقایسه‌ی کلی بین ابزارها در غلبه بر چالش های گوناگون

4.بحث و نتیجه‌گیری

نتایج بررسی مقالات نشان می‌دهد که ارتباطات بین سرویس‌ها در معماری میکروسرویس یکی از حیاتی‌ترین جنبه‌های طراحی و اجرای این معماری است. ابزارها و پروتکل‌های مختلفی برای مدیریت ارتباطات بین سرویس‌ها ارائه شده‌اند که هر کدام نقاط قوت و ضعف خاص خود را دارند. ابزارهای سنکرون مانند REST و gRPC در سناریوهای بلادرنگ با بار پایین عملکرد مناسبی دارند، اما در مواجهه با بارهای سنگین و نیاز به مقیاس‌پذیری دچار محدودیت می‌شوند. در مقابل، ابزارهای آسنکرون مانند Kafka و RabbitMQ با استفاده از معماری‌های انتشار-اشتراک و صف‌های پیام، توانسته‌اند تأخیر را کاهش داده و مقیاس‌پذیری سیستم‌های توزیع‌شده را بهبود بخشند. Kafka به‌ویژه در مدیریت جریان‌های داده با حجم بالا و نیازهای بلادرنگ برتری داشته و برای پردازش داده‌های کلان در محیط‌های پیچیده مانند اینترنت اشیا و محاسبات لبه‌ای مورد استفاده قرار گرفته است.

معماری‌های رویدادمحور نیز به‌عنوان راهکاری کارآمد برای کاهش وابستگی بین سرویس‌ها و بهبود مقیاس‌پذیری معرفی شده‌اند. این معماری‌ها با بهره‌گیری از ابزارهایی مانند Kafka و Redis Streams توانسته‌اند نیازهای سیستم‌های توزیع‌شده را در حوزه‌هایی مانند پردازش داده‌های حسگرها و تحلیل بلادرنگ داده‌ها برآورده کنند. همچنین، یکپارچگی این ابزارها با فناوری‌های یادگیری ماشین و هوش مصنوعی، مانندKafka-ML ، فرصت‌های جدیدی برای ایجاد سیستم‌های هوشمند مبتنی بر داده فراهم کرده است.

5.پیشنهادات و پژوهش‌های آتی

با وجود پیشرفت‌های صورت‌گرفته، همچنان چالش‌هایی در این زمینه وجود دارد که نیازمند تحقیق و توسعه بیشتر هستند. یکی از این چالش‌ها، مسئله بهینه‌سازی تأخیر در سیستم‌هایی با ترکیبی از ارتباطات سنکرون و آسنکرون است. استفاده همزمان از این دو نوع ارتباط می‌تواند به پیچیدگی‌های معماری و افزایش تأخیر منجر شود. تحقیق در زمینه مدل‌های ترکیبی و هوشمند برای مدیریت این پیچیدگی‌ها ضروری به نظر می‌رسد.

چالش دیگر، امنیت در ارتباطات بین سرویس‌ها است. اگرچه ابزارهایی مانند Kafka و RabbitMQ مکانیزم‌های امنیتی پیشرفته‌ای ارائه می‌دهند، اما مدیریت و پیکربندی این مکانیزم‌ها در محیط‌های توزیع‌شده با مقیاس بزرگ، همچنان دشوار است. تحقیقاتی که به ساده‌سازی مدیریت امنیت و ارائه روش‌های خودکار برای کشف و اصلاح آسیب‌پذیری‌ها بپردازند، می‌توانند ارزشمند باشند.

مقیاس‌پذیری و مدیریت منابع در محیط‌های ابر و لبه نیز یکی دیگر از مسائل مهم است. ابزارهایی مانند Kafka در مقیاس‌پذیری موفق عمل کرده‌اند، اما نیاز به بهینه‌سازی بیشتر در مدیریت منابع و کاهش مصرف CPU و حافظه وجود دارد. استفاده از فناوری‌های جدید مانند محاسبات مه یا الگوریتم‌های یادگیری ماشین برای بهینه‌سازی مصرف منابع، می‌تواند بهبودهای قابل‌توجهی در این زمینه ایجاد کند.

در نهایت، یکپارچگی ابزارهای مدیریت جریان داده با الگوریتم‌های پیشرفته تحلیل داده و یادگیری ماشین، مانند یادگیری آنلاین، نیازمند توجه بیشتری است. امروزه با پیشرفت چشم‌گیر علوم هوش مصنوعی، یادگیری ماشین و یادگیری عمیق و شبکه های عصبی مدل ها و راه‌حل‌های کارآمدی برای مسائل گوناگون مثل سیستم‌های توصیه‌گر، پیش‌بینی‌گر، بهینه سازی مثل بهینه سازی مصرف انرژی، بهینه‌سازی راندمان تولید و ...ارائه شده اند که با تجزیه و تحلیل داده‌های تولید شده از سیستم‌های دیگر کار می‌کنند. حال می توان سیستم هایی را توسعه داد که با به‌کارگیری این مدل‌ها و استفاده از این انبوه داده‌ها که می بایستی به طور ایمن، مطمئن، با تاخیر حداقلی و سایر ویژگی های ذکرشده را انتقال داد تا در قالب جریان‌های داده، ورودی مدل‌ها شوند و سیستمی بلادرنگ و زمان حقیقی را توسعه داد که با پردازش این داده‌ها می تواند منجر به نتایج بسیار مهمی و چشم‌گیری در پیشرفت و توسعه و ارتقا بهره‌گیرندگان از این سامانه شود. این مهم مادامی محقق می‌شود که با انتخاب طراحی صحیح، استراتژی‌های کارآمد، ابزارهای مناسب و پیاده‌سازی صحیح و نظرات کارشناسی و متقن انجام شود. تحقیقات آینده می‌توانند بر روی توسعه سیستم‌های بلادرنگی متمرکز شوند که از الگوریتم‌های پیشرفته و کارآمدی بهره می‌برند که در یک معماری نرم افزاری صحیح و با ابزارهای متناسب و طراحی کارشناسی شده، پیاده سازی شده‌اند تا نرم افزارهایی کارآمد برای پیش‌بینی، تحلیل و تصمیم‌گیری در لحظه توسعه داده‌شوند. چنین سیستم‌هایی می‌توانند تأثیر بزرگی بر صنایع مختلف، از جمله شهرهای هوشمند، بهبود بازدهی تولید، تجارت الکترونیک و ... داشته باشند.

این پیشنهادات می‌توانند به‌عنوان نقشه راهی برای توسعه سیستم‌های پیشرفته و رفع چالش‌های موجود در ارتباطات بین‌سرویسی در معماری میکروسرویس عمل کنند. با تکیه بر این پیشنهادات، می‌توان نه‌تنها چالش‌های موجود را کاهش داد، بلکه سیستم‌هایی با کارایی، امنیت، و انعطاف‌پذیری بالاتر ایجاد کرد که در مواجهه با نیازهای پیچیده و پویا، عملکردی بهینه داشته باشند.

6.مراجع و منابع

[1] Morgan Bruce, Paulo A. Pereira ،Microservices in Action ،Manning .2018 ،

[2] Neha Narkhede, Gwen Shapira, Todd Palino ،Kafka: The Definitive Guide: Real-Time Data and Stream Processing at Scale ،O'Reilly Media, Inc. .2017 ،

[3] J. Carnell ،Spring Microservices in Action ،Manning .2021 ،

[4] C. Walls ،Spring in Action, Sixth Edition ،Manning .2022 ،

[5] Işıl Karabey Aksakalli,Turgay Çelik,Ahmet Burak Can,Bedir Tekinerdogan ،"Deployment and communication patterns in microservice architectures: A systematic literature review "، *Systems & Software* .2021 ،المجلد 180 ،

[6] P. Badgujar ،"Implementing Event-Driven Architectures for Real-Time Insights "،*Journal of Technological Innovation* .2024 ،المجلد 5، رقم 1 ،

[7] Sidath Weerasinghe, Indika Perera ،"Optimized Strategy for Inter-Service Communication in Microservices "،*International Journal of Advanced Computer Science and Applications* ،المجلد 14pp. 272-279 .2023 ،رقم 2،

[8] K. S. E. Philippe Dobbelaere ،"Industry Paper: Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations "،*DEBS* ،pp. 227-238 .2017 ،

[9] Roman Wiatr, Renata Slota ،"Optimising Kafka for stream processing in latency sensitive systems تألیف7" ،*th International Young Scientist Conference on Computational Science* ،
.2018

[10] Md Showkat Hossain Chy ,Muhammad Ashfakur Rahman Arju ،"Comparative Evaluation of Java Virtual Machine-Based Message Queue Services: A Study on Kafka, Artemis, Pulsar, and RocketMQ "،*MDPI Electronics* ،p. 26 .2023 ،

[11] Benyamin Shafabakhsha, Robert Lagerström and Simon Hacks ،"Evaluating the Impact of Inter Process Communication in Microservice Architectures تألیف8" ،*th International Workshop on Quantitative Approaches to Software Quality* .2020 ،

[12] Sabrine Khriji, Yahia Benbelgacem,Rym Chéour ,Olfa Kanoun ,Dhouha El Houssaini ،
"Design and implementation of a cloud-based event-driven architecture for real-time data processing in wireless sensor networks "،*The Journal of Supercomputing* .2021 ،

[13] C. Lekkala ،"-Designing High-performance, Scalable Kafka Clusters for Real-time Data Streaming "،*European Journal of Advances in Engineering and Technology* ،pp. 76-82 .2021 ،

[14] Ajay Bandi, Julio Ariel Hurtado ،"Big Data Streaming Architecture for Edge Computing Using Kafka and Rockset تألیف" ،*Proceedings of the Fifth International Conference on Computing Methodologies and Communication* .2021 ،

[15] Cristian Martín, Prof. Dr. Peter Langendörfer, Pouya Soltani Zarrin, Manuel Díaz, Bartolomé Rubio ،"Kafka-ML: Connecting the data stream with ML/AI frameworks "،*Future Generation Computer Systems* ،pp. 15-33 .2022 ،

[16] SEDA KUL, ISABEK TASHIEV , ALI ŞENTAŞ ,AHMET SAYAR ،"Event-Based Microservices With Apache Kafka Streams: A Real-Time Vehicle Detection System Based on Type, Color, and Speed Attributes "،*IEEE Access* ،المجلد 9 ،pp. 83137-83148 .2021 ،